

Почему ChatGPT не может посчитать, сколько букв «r» в слове strawberry?

Спросите любую LLM — половина случаев ответит «2».

Ответ:

Вы видите:

s · t · r · a · w · b · e · r · r · y (10 букв)

AI видит:

[st]

[raw]

[berry]

(3 токена)

Слово strawberry в токенизаторе o200k_base (GPT-4o, Claude 4.x) разрезается на 3 токена. Модель не «видит» буквы.

Как работают современные большие модели

4 этапа inference: токенизация · эмбеддинг · внимание · сэмплинг



Сегодня углубляем слой «модель» из четырёх слоёв Лекции 1

(Лекция 1 §3.2)

Приложение

Агент

Чат

МОДЕЛЬ — углубляем сегодня

Что мы знаем (Лекция 1 §3.2):

Модель = stateless inference. Вход — данные, выход — предсказание. Между вызовами памяти нет.

Что узнаем сегодня:

Что внутри inference. 4 этапа:

токенизация → эмбединг → внимание → сэмплинг

Главный вопрос лекции

«Что происходит внутри LLM между моим запросом и ответом — и какие из этих механизмов меняют, как я её использую?»

3 ответа — финал лекции

1

Почему промпт с ролью работает лучше пустого?

→ Раздел 3 — внимание

2

Почему AI плохо считает буквы?

→ Раздел 1 — токенизация

3

Почему один запрос даёт разные ответы?

→ Раздел 4 — сэмплинг

Токен — id из словаря модели. Не буква и не слово.

Статистически частая подпоследовательность

Пример 1.

cat

→

[cat]

1 токен / 1 id

Пример 2.

tokenization

→

[token]

[ization]

2 токена

Пример 3.

клубника

→

[к]

[луб]

[ника]

3 токена ·
o200k_base

В среднем: 1 токен ≈ 4 символа в EN ≈ 2 символа в RU

Подумайте 15 сек: «сильнее» — 1, 2 или 3 токена? (Проверить через tiktokenizer)

BPE — компромисс между алфавитом и словарём

Словарь строится один раз перед обучением; в inference — lookup

Before (обучающий корпус)

- low
- lower
- newest
- widest



After (BPE-словарь)

- low
- er
- new
- est
- wid

BPE-словарь строится один раз до обучения. В inference — lookup готовых правил, не runtime-вычисление.

AI ошибается в «сколько r в strawberry» — слова не из букв, а из 2-3 токенов

strawberry в o200k_base (GPT-4o)

10 букв — то, что видит человек:

s t r a w b e r r y



3 токена — то, что видит модель:

st raw berry

Модель видит 3 единицы — не 10 букв

Подсчёт символов

«Сколько r в strawberry?» — ломается систематически, неочевидно для пользователя.

Опечатки

methodology → другие токены, чем methodology.
Маленькая опечатка → большой сдвиг в ответе.

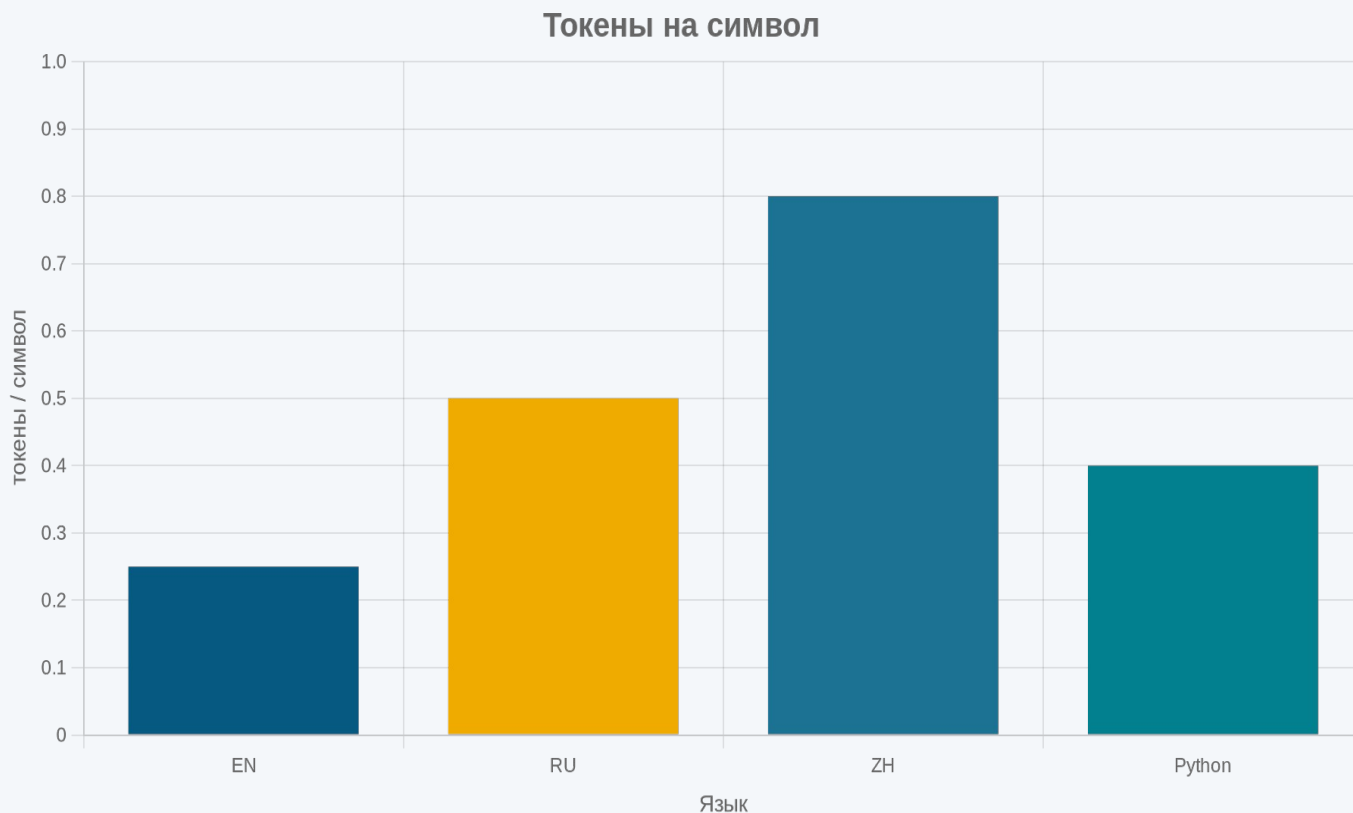
Регистр и пробелы

cat, `cat`, Cat, CAT — разные токены, разные id.

Для побитово-точных операций — внешний инструмент (Python, regex), не чистый LLM-инференс.

Один и тот же текст по-русски стоит в 2× дороже, чем по-английски

Токены на символ — следствие распределения языков в обучающем корпусе



Ориентир токены/символ

Английский	0.25
Русский (gold)	0.50
Китайский	0.80
Python-код	0.40

API-стоимость RU $\approx$ 2× от EN. Для batch — переводить в EN, если допустимо.

Каждому токену сопоставлен вектор — выучен на тренировке, фиксирован

Эмбе́ддинг: id → выученный вектор

[КОТ]

id = 47284

lookup в
embedding table →

[0.21, -0.45, 0.88, ..., 0.13]

несколько сотен — несколько тысяч измерений
выучен на тренировке, в inference фиксирован

Геометрическая близость = семантическая близость

Размерности (ориентир)

text-embedding-3-small

1536 dim

text-embedding-3-large

3072 dim

Внутренний эмбе́ддинг flagship LLM

тысячи dim

Геометрическая близость векторов = семантическая близость токенов.

«Кот» близко к «собаке» — выучилось из контекстов корпуса.

Близость в пространстве эмбедингов = семантическая близость

В 2026 это работает на уровне предложений, не только слов

Cosine similarity — 5 предложений

Cosine = угол между векторами, [0, 1]; ближе к 1 — более похожи

	SSL	HTTPS	React-комп.	React-прил.	Борщ
SSL	1.00	0.85	0.18	0.20	0.08
HTTPS	0.85	1.00	0.22	0.19	0.07
React-комп.	0.18	0.22	1.00	0.78	0.12
React-прил.	0.20	0.19	0.78	1.00	0.10
Борщ	0.08	0.07	0.12	0.10	1.00

Cosine similarity — мера угла между векторами; диапазон $[-1, 1]$, ближе к 1 — более похожи.

Числа illustrative; воспроизводимы на `sentence-transformers/all-MiniLM-L6-v2` (384-dim) или `OpenAI text-embedding-3-small` (1536-dim).

Эмбединги дают similarity, clustering и search — основу RAG

Три практических применения, настраиваемых над одной embedding-таблицей



Similarity

Поиск похожих

Похожие тикеты в support,
кейсы в юр-базе,
резюме в HR-системе.



Clustering

Кластеризация

k-means — анализ
жалоб клиентов,
тематика корпусов.



Search

Семантический поиск

**Запрос → top-K
похожих документов.**

→ **Основа RAG (Лекция 3)**

Semantic search находит то, что full-text пропустит

Запрос

клубника

Full-text (Elasticsearch, Lucene)

- ✓ клубника
- ✓ клубники (стемминг)
- ✓ клубнику
- ✗ *strawberry*
- ✗ *ягода*
- ✗ *лесная земляника*

Semantic (эмбединги + ближайшие соседи)

- ✓ клубника — *точное*
- ✓ клубники — *морфология*
- ✓ **strawberry** — *cross-lang*
- ✓ ягода — *родовое*
- ✓ **лесная земляника** — *близкий смысл*
- ✓ ... — *ещё близкие*

Раздел 3

Механизм внимания

Как модель решает, что важно сейчас

0 Открытие

1 Токены

2 Эмбединги

3 Внимание

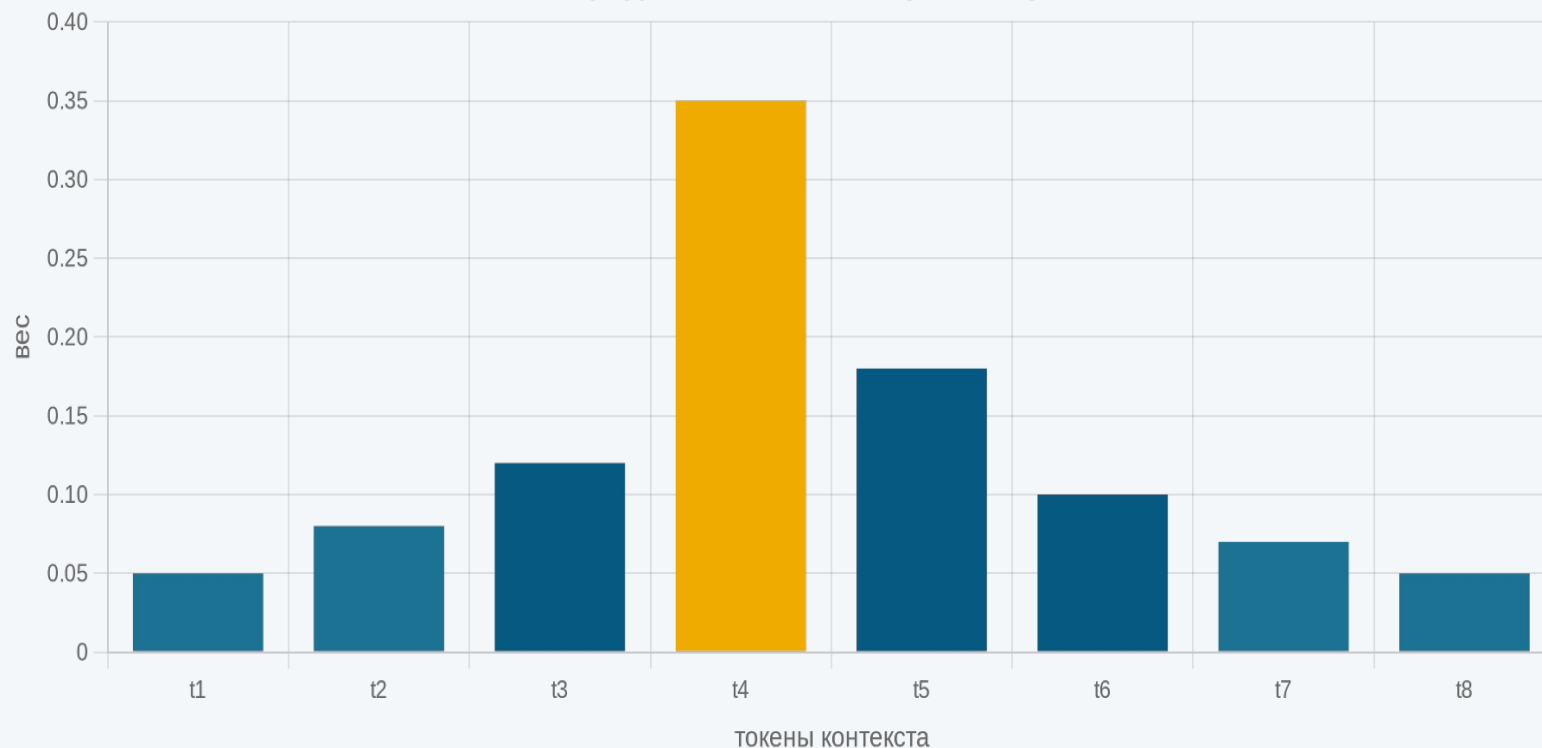
4 Сэмплинг

5 Финал

Attention выдаёт распределение весов на все токены контекста (сумма = 1)

Какие токены сейчас важны для предсказания следующего

Распределение attention (sum = 1)

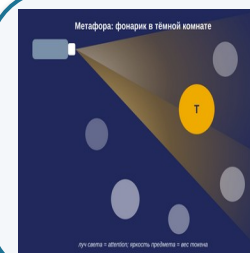


Распределение весов на токенах контекста — основное содержание

Без формул. Multi-head, Q/K/V — доп. чтение (Vaswani et al. 2017).

3 свойства

- 1 На вход — все токены контекста.
- 2 На выходе — распределение, $\Sigma = 1$.
- 3 Пересчитывается на каждом шаге.



Метафора: фонарик в тёмной комнате — модель «подсвечивает» одни токены ярче других.

Role-токены получают повышенный вес в attention

Часть A — рабочий пример; часть B — эффект роли (1-е из 3 «почему»)

A. Worked example — куда смотрит «она»

«Кот съел мышь, потому что она была голодна»

она → мышь

толстая · главный вес

она → была

средняя

она → голодна

тонкая

Упрощение: реальный attention map — сотни связей. Модель смотрит статистически, не делает грамматический разбор.

Без роли

«Объясни асинхронность»

→ *обобщённый ответ*

(низкий вес role-токенов в attention)

С ролью

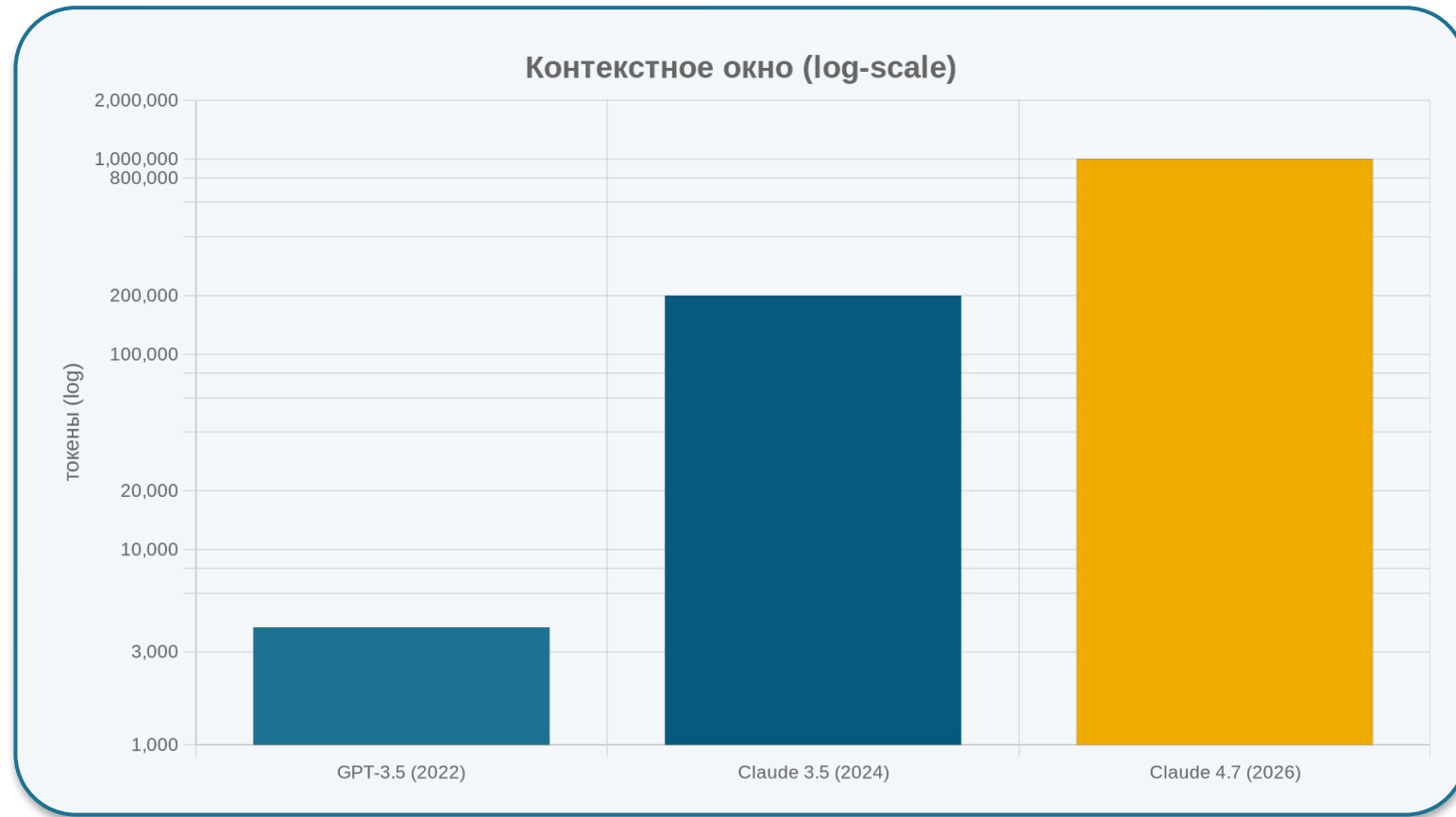
«Ты **эксперт по Python**. Объясни асинхронность **джуниору**.»

→ **role-токены подсвечены**

(высокий вес в attention)

Контекстное окно — физический предел того, сколько модель видит одновременно

Эволюция *context window* + квадратичная стоимость *attention*



Эволюция и стоимость

2022 → 2026: ×250 рост

Темп: ×10 / 1-2 года

Cost N²: 1M ≈ 16× от 100k

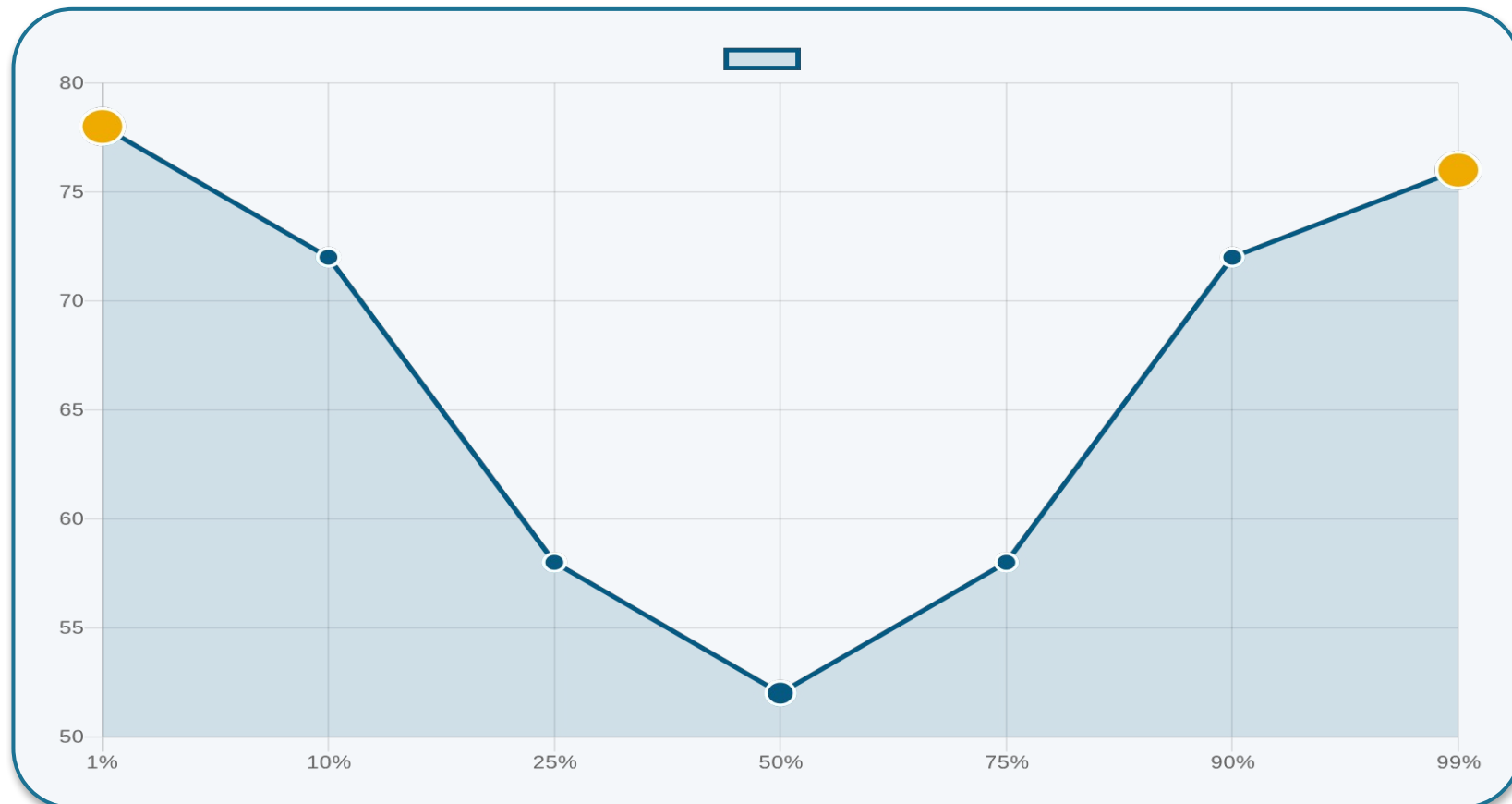
Архитектура: ванильная attention

N²

Стоимость attention растёт квадратично от длины. 1M ≈ 16× дороже 100k — production-pricing с batching; чистая N²-теория дала бы 100×.

Большое контекстное окно $\neq$ хорошее использование контекста

Lost-in-the-middle effect — модель забывает середину



Эксперимент

Факт вставлен в позицию X (начало / середина / конец) 100k-контекста. Модель отвечает на факт.

Результаты

Начало: ~75%

Середина: ~50%

Конец: ~75%

*Liu et al. 2023.
Lost in the Middle.*

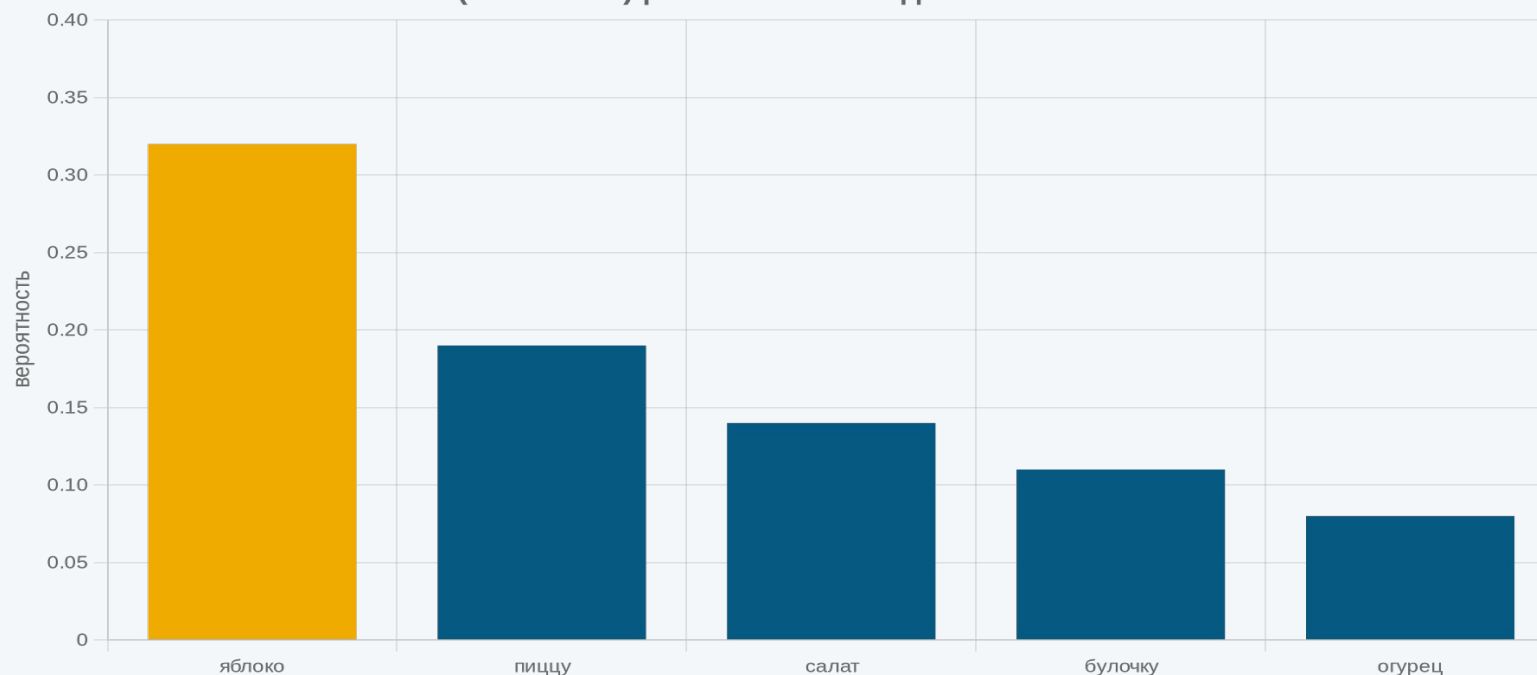
Инженерный вывод: важное помещайте в начало или в конец промпта, не в середину.

На каждом шаге модель выдаёт распределение вероятностей на все токены — выбирает один

Контекст

«Сегодня я съел ...»

P(next token) | контекст: «Сегодня я съел...»



Топ-5 кандидатов

яблоко	0.32
пиццу	0.19
салат	0.14
булочку	0.11
огурец	0.08

... остальные ~200k токенов:
каждый < 0.05

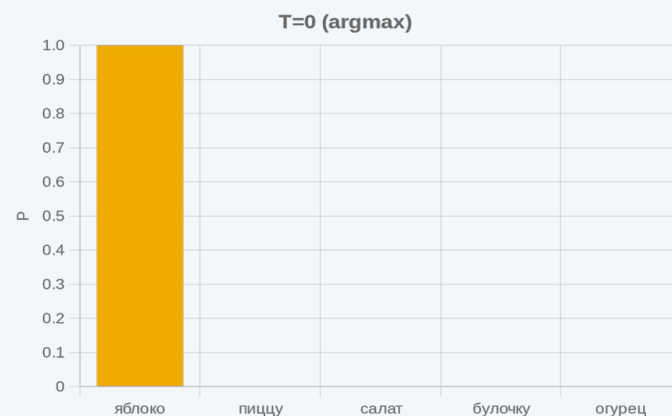
$$\Sigma = 1$$

Сэмплинг = правило, по которому из распределения выбирается один токен. Дальше — температура.

Температура: насколько острым будет выбор

$T = 0$ (*argmax*) · $T = 1.0$ (стандарт) · $T = 2.0$ (хаос)

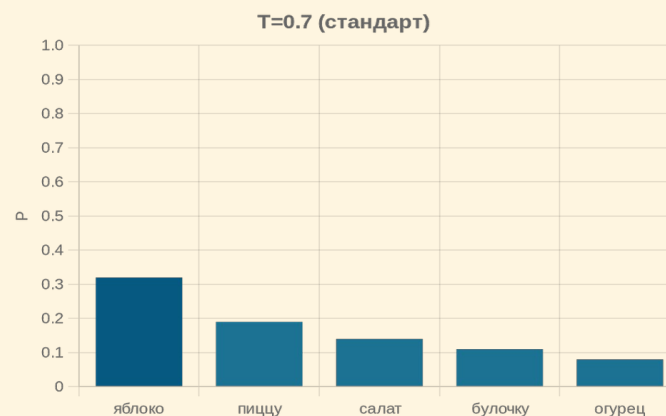
$T = 0$ · *argmax*



Детерминированный
выбор — яблоко.

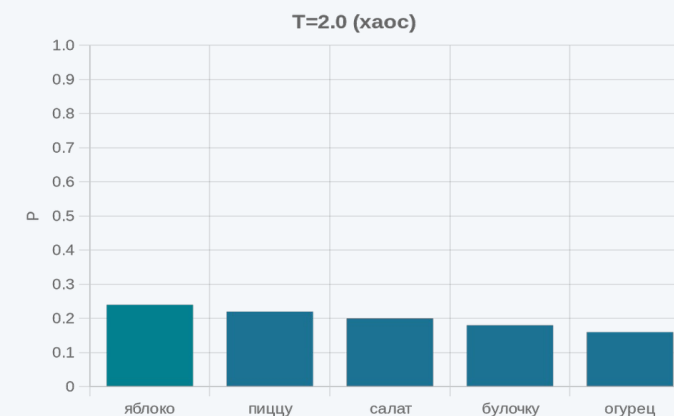
10 запусков → одинаково.

$T = 1.0$ · стандарт



Сэмплирование
пропорционально P .
Естественная вариативность.
($T = 0.7$ — *consensus* для чата)

$T = 2.0$ · хаос



Распределение сглажено;
часто выбираются
неожиданные варианты.

Альтернативные ручки: top-p (nucleus) — отрезает редкие токены по Σ ; top-k — по числу кандидатов. Достаточно T для start.

4 ручки API под задачу: temperature, top_p, max_tokens, system prompt

Подобрать параметры под сценарий обоснованно

Сценарий	temperature	top_p	max_tokens	system_prompt
Классификация / точное извлечение	0	—	50–200	Минимальный, со схемой выхода
Кодогенерация	0.2–0.3	0.9	1000+	Роль + контекст репозитория
Чат-объяснение пользователю	0.7	0.9	500–1000	Роль + описание аудитории
Творческое письмо	0.9–1.2	0.95	2000+	Роль + описание стиля

T = 0 практически детерминирует выбор; в production возможна микро-вариативность из-за batching — для большинства задач игнорируема.

Цикл: предсказали токен → добавили в контекст → предсказываем следующий

Авторегрессионная генерация — длинный ответ из stateless-вызовов



Каждый шаг — stateless. «Память» одного ответа несёт сам контекст, не модель.

Inference одинаков локально и в облаке — но размер модели определяет качество

Архитектурно — тот же конвейер. Различия — в размере и среде.

Local (Ollama, llama.cpp, LM Studio)

Размер: 1–13B параметров

- Qwen 2.5 1.5B • Llama 3.2 1B
- Llama 3.1 8B • Mistral 7B

- **Приватность** *запросы не уходят провайдеру*
- **Скорость** *медленнее на consumer hardware*
- **Контекст** *ограниченное окно*
- **Цена** *0 за токен (своё железо)*

Cloud (OpenAI, Anthropic, Yandex, Сбер)

Размер: 200B+ параметров

- GPT-5, Claude 4.7
- YandexGPT, GigaChat
- Gemini

- **Качество** *лучше на сложных задачах*
- **Задержка** *200–500 мс*
- **Контекст** *до 1M токенов*
- **Цена** *оплата за токены, RU ≈ 2× EN*

4 этапа inference сложились в конвейер

Тот же чёрный ящик из Лекции 1 §3.2 — теперь распакован



Лекция 1 §3.2 называла этот конвейер «inference моделью» — чёрным ящиком. Теперь он перестал быть чёрным.

3 промиса Лекции 1 — 3 ответа из Лекции 2

Payoff Лекции 1 §5.3 — связь обещаний и механизмов

1

Почему промпт с ролью работает лучше пустого?

На уровне attention role-токены получают высокий вес — модель опирается на них при выборе следующих токенов.

2

Почему AI плохо считает буквы?

Токенизатор объединяет буквы в токены. strawberry — 3 токена, не 10 букв. Модель видит токены, не буквы.

3

Почему один и тот же запрос даёт разные ответы?

Сэмплинг — стохастический выбор из распределения при $T > 0$. Каждый запуск может выбрать разный токен.

LLM — не всегда правильный инструмент. Дерево решений: когда не LLM

Когда LLM — не правильный инструмент?



Фиксированные классы

Классификация на маленьком наборе категорий (5–20)?

→ Классический ML
лог. регрессия, XGBoost,
LightGBM, дообученный BERT



Интерпретируемость

*Нужна интерпретируемость
(финансы, медицина, страхование)?*

→ Прозрачные методы
лог. регрессия + важность,
деревья решений, правила



Скорость отклика

*Время отклика < 100 мс критично
(антифрод, устройство
пользователя)?*

→ Специализированная
маленькая модель
(не LLM ≥ 200 мс)

Иначе — LLM подходит (chat, RAG, generation, многошаговое рассуждение)

Attention статистически смотрит на токены — не понимает причинности

AI считает корреляции в данных, не строит каузальный граф



Человек

«X произошло, потому что Y»

Модель причинности — строит механизмы.

*Опирается на физическую интуицию,
доменные знания, знание механизмов мира.*



AI (через attention)

«X следует за Y в данных»

*Статистическая корреляция, не
причинность.*

*Замечает паттерн «X и Y часто
соседствуют» в обучающих данных —
корреляция, не каузальный граф.*

Инженерный вывод: для причинных выводов привлекайте domain-эксперта или causal-методы.

Принесите: 1 запрос × 3 температуры × 3 запуска × анализ

ДЗ к Семинару 2 — apply температуры на своей задаче



Шаг 1

Возьмите типовую задачу из своей предметной области.

Конкретный воспроизводимый запрос (не «помоги думать»).



Шаг 2

Запустите в playground на 3 температурах.

$T = 0 \cdot T = 0.7 \cdot T = 1.5$
по 3 запуска каждой
(для оценки variance)



Шаг 3

Принесите одностраничный разбор (1 A4).

Что изменилось / осталось / какую T для production.

Playground: Hugging Face Inference Playground

Модель: Meta-Llama-3-8B-Instruct (apples-to-apples)

Fallback: Together.ai / Ollama локально · НЕ подойдут: ChatGPT Free, Claude.ai

БОНУС

«Сколько p в "строгая регуляризация"» × 3 модели. Объяснить через токенизацию.

Лекция 3: «Агенты, RAG, API — как AI выходит за пределы чата»

Все 4 концепции надстраиваются над одним проходом inference



RAG

Retrieval-Augmented Generation

близость эмбедингов + LLM → ответ из вашей базы



Инструменты / Вызов функций

структурированный JSON

LLM генерирует вызов → выполняет внешняя система
→ результат возвращается



MCP

Model Context Protocol

Открытый стандарт подключения инструментов
(Anthropic, 2024)



Цикл агента

действуй → наблюдай → корректируй

Модель решает действие, видит результат,
корректирует план

Q&A

До 5 минут на вопросы в зале. Дополнительные — на Семинар 2 или e-mail.